# TECHNICAL SPECIFICATION DOCUMENT

## Wedding Guest Management & RSVP Automation Platform

*v1.0 · Implementation-Ready Specification · Full-Stack Engineering Team*

| Stack Component | Technology |
|---|---|
| Frontend | React 18 + TypeScript + Tailwind CSS |
| Backend | Node.js (NestJS) + REST API |
| Database | PostgreSQL 15 + Prisma ORM |
| Automation | n8n (self-hosted) |
| Messaging | WhatsApp Business API (Meta Cloud API / Twilio) |
| Queue | BullMQ + Redis |
| Auth | JWT + Refresh Tokens + OTP |
| Hosting | Docker + Docker Compose (cloud-deployable) |

# PRODUCT VISION & PURPOSE

*What we're building, why it matters, and who it's for*

## The Problem We're Solving

Weddings in India — especially large-scale ones in Kerala — are not small, intimate gatherings. They are elaborate, multi-day social events involving hundreds of families, complex guest hierarchies, and an intricate web of relationships. A typical Kerala wedding has between 500 and 2,000 guests. Managing

who is invited, who has confirmed, who is bringing how many family members, and who still hasn't responded is a logistical challenge that no spreadsheet or WhatsApp group can handle well.

Today, wedding organizers and planners manage this entire process manually. They maintain Excel sheets that become unmanageable. They chase guests through personal WhatsApp messages sent one by one. They have no idea of their actual headcount until the day before the event, which causes catering overruns, seating chaos, and avoidable stress for everyone involved. There is no single source of truth.

**The Core Pain:** A wedding planner managing a 1,200-guest Kerala wedding is currently juggling 4 Excel files, 6 WhatsApp groups, and a notepad. They have zero real-time visibility into confirmed attendance. This platform eliminates all of that.

## Who Uses This Platform

| User | Who They Are | What They Need |
|---|---|---|
| Wedding Planner / Organiser | Professional coordinator managing the entire event. Could be handling multiple weddings simultaneously. | A command center: one dashboard showing total confirmed attendance, pending RSVPs, campaign status, and headcount in real time. |
| Bride's Family Coordinator | A trusted family member or assigned point of contact on the bride's side. May not be tech-savvy. | Simple access to only their side's guest list. Ability to see who has confirmed and follow up with pending guests. |
| Groom's Family Coordinator | Same role, opposite side. Often operating independently from the bride's team. | Identical to above but scoped to groom side. No visibility into bride side data unless granted. |
| Wedding Guest | The person receiving the invitation. Often an older relative in Kerala who uses WhatsApp but may not have email. | A simple WhatsApp message they can reply to. No app download. No form to fill. Just reply YES or NO. |

## The Solution: What This Platform Does

This platform is a purpose-built SaaS CRM for wedding guest management and RSVP automation. It replaces spreadsheets, manual WhatsApp messages, and guesswork with a structured, automated system that gives every stakeholder exactly what they need — nothing more, nothing less.

**01**

### Guest List Management

Import up to 2,000 guests via CSV or add them manually. Organise guests into family groups, tag them (VIP, Outstation, Vegetarian), and split them between Bride Side and Groom Side. Every guest has a profile, a family, and a side.

**02**

### WhatsApp Invitation Campaigns

Create personalised WhatsApp message campaigns using Meta-approved templates. Each message is addressed to the guest by name, includes their family name, event details, venue, and a clear call to action. Campaigns can target one side or both.

**03**

### Automated RSVP Collection

When a guest replies 'Yes', 'No', or 'Coming with family' on WhatsApp, the system captures and classifies that response automatically. No human needs to read and log replies. The response is stored, the RSVP record is updated, and a follow-up workflow fires.

**04**

### Family-Level Headcount Tracking

RSVPs are tracked at both the individual and family-group level. If 'Sharma Family' has 5 members and 3 confirm, the dashboard shows partial confirmation. The confirmed headcount (for catering and seating) is always accurate and live.

**05**

### Intelligent Automation (n8n)

The platform automatically sends reminders to guests who haven't responded after a configurable number of days. Once a guest confirms, a thank-you message goes out. The day before each event, confirmed guests receive a reminder with venue details.

**06**

### Bride & Groom Side Separation

This is a first-class architectural feature, not a filter. Every guest, campaign, and RSVP lives on one side. Coordinators only see their side's data. Analytics show totals broken down by side. This mirrors how large Indian weddings are actually organised.

**07**

### Real-Time Dashboard

A live dashboard shows total invited, confirmed, declined, pending, and total headcount — globally and per side. Organizers know at any moment exactly how many people are coming to each event. No more day-before panic.

**08**

### Message Delivery Tracking

Every outbound message has a tracked status: Queued → Sent → Delivered → Read → Replied. Failed messages are flagged with error codes. The system handles WhatsApp rate limits automatically via a queue, so bulk sends of 1,000+ messages work reliably.

## Why We Built It This Way

Every technology decision in this platform was made in response to a real constraint of the Indian wedding context. Below is the reasoning behind each major choice, written so the development team understands not just what to build, but why.

| Decision | Why |
|---|---|
| WhatsApp as the primary communication channel | Email open rates for Indian wedding guests hover near zero. WhatsApp has near-universal penetration in India, including |

| Decision | Why |
|---|---|
| | among older generations. A 65-year-old uncle in Kerala is on WhatsApp. He is not checking email. This is the only channel that reaches every demographic. |
| Reply-based RSVP instead of a web form | A web form requires the guest to open a browser, find the link, and fill out a form. A WhatsApp reply requires the guest to type 'Yes'. The simpler the action, the higher the response rate. The system is designed to meet guests where they are. |
| Bride Side / Groom Side as a core data layer | In Kerala and across India, the bride's family and groom's family operate as two distinct social networks. Their guest lists don't overlap. Their coordinators are different people. Mixing them into a single undifferentiated list creates confusion and friction. The architecture reflects social reality. |
| Family-group tracking instead of individual-only | Indian RSVPs are rarely individual. When you invite 'the Sharma family', you mean all of them. The catering order is by head count, not by individual confirmation. Tracking at the family level gives planners the numbers they actually need. |
| NestJS (structured) over plain Express | This platform has multiple complex modules (auth, guests, campaigns, webhooks, RSVP, stats) that need clear separation. NestJS enforces module boundaries, dependency injection, and guard-based middleware — which maps directly to our RBAC and SideGuard needs. It scales with the team. |
| BullMQ queue for WhatsApp sends | Sending 1,000 WhatsApp messages synchronously in a loop will hit Meta's rate limits and timeout the HTTP request. BullMQ processes messages as background jobs with configurable concurrency, rate limiting, and retry logic. It's the only correct way to do bulk messaging. |
| n8n for automation instead of cron jobs in code | Automation workflows (reminders, thank-you messages, pre-event nudges) change frequently. Event-driven timing, retry logic, and conditional branching are complex to maintain in code. n8n gives the team a visual, configurable workflow engine that non-engineers can adjust without a deployment. |
| PostgreSQL row-level isolation over separate databases per tenant | Each wedding is a tenant. A separate database per wedding is operationally expensive and unnecessary at this scale. Row-level isolation with indexed wedding_id columns gives strong data separation with much lower infrastructure cost. This is standard SaaS practice. |

## What Does Success Look Like?

When this platform is fully built and deployed, the following outcomes define success. These should be the developer's north star when making implementation decisions — any feature that moves these metrics forward is worth building; any feature that doesn't should be questioned.

| Success Metric | Target | Why It Matters |
|---|---|---|
| RSVP Response Rate | > 80% of invited guests respond | Higher than manual follow-up (typically 40-50%). Automation removes the bottleneck. |

| Success Metric | Target | Why It Matters |
|---|---|---|
| Time to Headcount Clarity | Known 7 days before event | Currently takes until 1-2 days before. Caterers need 5-7 days lead time. |
| Campaign Send Reliability | > 99% of queued messages sent | A failed invitation is a missed guest. Queue + retry must be bulletproof. |
| Organiser Setup Time | < 30 minutes from signup to first campaign sent | If onboarding is slow, planners abandon the tool. Wizard must be frictionless. |
| Guest Effort to RSVP | 1 WhatsApp reply | Zero friction. Guest should not need to download anything or click a link. |
| Data Accuracy | Zero duplicate guests per wedding | One phone number = one guest record. Deduplication must be enforced at DB level. |
| Side Isolation Correctness | Zero data leaks between sides | A Groom-side coordinator must never see Bride-side guest data. Non-negotiable. |

## A Note to the Development Team

**Read This First:** Every section of this specification exists because a real problem demanded a real solution. When you're implementing the SideGuard, remember it's not bureaucracy — it's protecting a groom-side coordinator from accidentally seeing confidential bride-family data. When you're building the RSVP parser, remember it's replacing a human who was manually reading and logging 500 WhatsApp replies. Build with that context in mind.

The sections that follow contain the complete technical blueprint: system architecture, database schema, API contracts, WhatsApp integration details, n8n workflow designs, and edge case handling. Everything a full-stack team needs to begin development immediately. Read the product vision above before diving into the technical detail — it will make every decision in the spec make more sense.

# 1. SYSTEM ARCHITECTURE

## 1.1 High-Level Architecture Overview

The platform is a multi-tenant SaaS application. Each wedding is an isolated tenant. Within a wedding, all data is partitioned by wedding_side (Bride / Groom). The architecture is composed of five primary layers:

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENT LAYER                          │
│   React 18 SPA  —  Axios  —  Zustand State  —  Side Context  │
└──────────────────────────────────────────────────────────────┘
                            │ HTTPS / REST
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                        API GATEWAY (NestJS)                  │
│   JWT Guard  —  RBAC Guard  —  Side Guard  —  Rate Limiter   │
```

```
│   Modules: Auth | Wedding | Side | Guest | Campaign | RSVP | Stats   │
└──────────────────────────────────────────────────────────────────────┘
      │ Prisma ORM                                      │ BullMQ Jobs
      ▼                                                 ▼
┌──────────────────────────┐                  ┌──────────────────────────┐
│  PostgreSQL 15           │                  │  Redis + BullMQ Queue    │
│  Multi-tenant schema     │                  │  whatsapp-send queue     │
│  Row-level isolation     │                  │  reminder-trigger queue  │
└──────────────────────────┘                  └──────────────────────────┘
                                                        │ Worker
                                                        ▼
                                              ┌──────────────────────────┐
                                              │  WhatsApp Worker         │
                                              │  Meta Cloud API / Twilio │
                                              └──────────────────────────┘
                                                        │ Webhook
                                                        ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          n8n Automation Engine                       │
│   Workflows: RSVP Follow-up │ Reminders │ Thank-You │ No-Response     │
│   HTTP Trigger ◀── NestJS event emitter ──▶ n8n REST calls NestJS    │
└──────────────────────────────────────────────────────────────────────┘
```

## 1.2 Data Flow: End-to-End Guest Journey

The following describes the complete data lifecycle from guest upload through RSVP capture to dashboard analytics:

| Step | Actor | Action | System |
|---|---|---|---|
| 1 | Organiser | CSV upload or manual guest entry | React → POST /guests/bulk |
| 2 | Backend | Validate, deduplicate, assign side & family | NestJS GuestService |
| 3 | Organiser | Create campaign, select template, assign side | React → POST /campaigns |
| 4 | Organiser | Launch campaign | POST /campaigns/:id/launch |
| 5 | Worker | Dequeue & send WhatsApp messages | BullMQ → WhatsApp API |
| 6 | WhatsApp | Delivers message to guest | Meta Cloud API |
| 7 | Guest | Replies to WhatsApp message | WhatsApp → Webhook POST |
| 8 | Backend | Parse RSVP response, store structured data | NestJS RSVPService |
| 9 | n8n | Trigger follow-up workflows based on response | n8n → NestJS Events |
| 10 | Dashboard | Real-time analytics aggregated by side | GET /stats/dashboard |

# 2. USER ROLES & PERMISSIONS (RBAC)

## 2.1 Role Definitions

The platform implements four roles with granular side-level scoping:

| Role | Scope | Description |
|---|---|---|
| SUPER_ADMIN | Platform | Cifar/platform operator. Access to all weddings and tenants. |
| WEDDING_ADMIN | Wedding | Full access to all data within a single wedding project. |
| SIDE_MANAGER | Side-specific | Access restricted to one side (Bride OR Groom) only. |
| VIEWER | Wedding/Side | Read-only access. Cannot create, edit or send campaigns. |

## 2.2 Permission Matrix

| Permission | SUPER_ADMIN | WEDDING_ADMIN | SIDE_MANAGER | VIEWER |
|---|---|---|---|---|
| Create Wedding | ✅ | ✅ (own only) | ❌ | ❌ |
| Manage All Guests | ✅ | ✅ | Own side only | ❌ |
| Create Campaign | ✅ | ✅ | Own side only | ❌ |
| Launch Campaign | ✅ | ✅ | Own side only | ❌ |
| View RSVP (all sides) | ✅ | ✅ | ❌ | ❌ |
| View RSVP (own side) | ✅ | ✅ | ✅ | ✅ |
| Configure WhatsApp | ✅ | ✅ | ❌ | ❌ |
| Manage Team Members | ✅ | ✅ | ❌ | ❌ |
| View Dashboard | ✅ | ✅ (all sides) | Own side only | Own side only |
| Export Data | ✅ | ✅ | Own side only | ❌ |

## 2.3 Side-Level Data Isolation (Architecture Layer)

**CRITICAL:** Side isolation is enforced at the service layer via a SideGuard NestJS interceptor, NOT just in UI. Every query to guests, campaigns, RSVP data, and analytics is automatically scoped to the user's assigned side. This cannot be bypassed by frontend manipulation.

Implementation pattern for SideGuard:

```
// side.guard.ts
@Injectable()
export class SideGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const req = context.switchToHttp().getRequest();
    const user = req.user; // from JWT
    const requestedSideId = req.params.sideId || req.body.sideId || req.query.sideId;
```

```
    if (user.role === Role.WEDDING_ADMIN) return true; // full access

    if (user.role === Role.SIDE_MANAGER) {
      // user.assignedSideIds is an array from JWT payload
      if (requestedSideId && !user.assignedSideIds.includes(requestedSideId)) {
        throw new ForbiddenException('You do not have access to this side.');
      }
    }
    return true;
  }
}
```

# 3. FEATURE MODULES — DETAILED SPECIFICATION

## 3.1 Authentication & Onboarding Module

### 3.1.1 Login Flow (Email + OTP)

1. User submits email → POST /auth/request-otp
2. Backend generates 6-digit OTP, stores hashed OTP + expiry in Redis (TTL: 10 min)
3. OTP sent via email (Nodemailer / SendGrid)
4. User submits OTP → POST /auth/verify-otp
5. Backend verifies OTP hash, issues JWT access token (15 min) + refresh token (7 days)
6. Refresh token stored in HttpOnly cookie; access token in memory (not localStorage)

### 3.1.2 Wedding Project Creation

On first login, user is redirected to /setup. This creates:

- A wedding record with basic details (couple names, date, venue)
- Two wedding_sides records: { name: 'bride', display_name: 'Bride Side' } and { name: 'groom', display_name: 'Groom Side' }
- User assigned WEDDING_ADMIN role for the wedding
- Default WhatsApp config slot created (unconfigured)

### 3.1.3 Team Invitation

- WEDDING_ADMIN sends invite via email (POST /team/invite)
- Invite record created with: email, role, assigned_side_ids[], token (UUID), expires_at
- Invitee clicks link → /accept-invite?token=<uuid>
- If new user: prompt to set name; OTP verification; account created
- If existing user: directly assigned to wedding with specified role and sides

## 3.2 Wedding & Event Management Module

### 3.2.1 Wedding Fields

| Field | Type | Notes |
|---|---|---|
| id | UUID | Primary key |
| tenant_slug | VARCHAR | URL-safe slug e.g. 'rahul-priya-2025' |
| bride_name | VARCHAR | |
| groom_name | VARCHAR | |
| wedding_date | DATE | |
| venue | TEXT | Full venue address |
| created_by | UUID FK | → users.id |
| whatsapp_config | JSONB | API keys, phone_number_id, etc. |
| is_active | BOOLEAN | Soft-enable/disable |

### 3.2.2 Event Fields

A wedding can have multiple events (engagement, mehndi, haldi, wedding, reception, etc.):

| Field | Type | Notes |
|---|---|---|
| id | UUID | PK |
| wedding_id | UUID FK | → weddings.id |
| side_id | UUID FK | NULL = applies to both sides |
| name | VARCHAR | e.g. 'Wedding Ceremony' |
| event_date | DATE | |
| event_time | TIME | |
| venue | TEXT | |
| maps_link | TEXT | Google Maps URL |
| dress_code | VARCHAR | Optional |
| notes | TEXT | Optional instructions |

## 3.3 Wedding Sides Module (CORE ARCHITECTURE)

**ARCHITECTURE NOTE:** wedding_sides is not a UI tag. It is a first-class database entity that drives all data partitioning. Every guest, campaign, RSVP response, and analytics query carries a side_id foreign key.

### 3.3.1 wedding_sides Table

```
CREATE TABLE wedding_sides (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id    UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  name          VARCHAR(20) NOT NULL,        -- 'bride' | 'groom' | 'custom'
  display_name  VARCHAR(100) NOT NULL,       -- 'Bride Side' | 'Groom Side'
```

```
  color_hex     VARCHAR(7) DEFAULT '#0E7490',-- UI accent colour for this side
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(wedding_id, name)
);
```

### 3.3.2 Global Side Filter (Frontend)

A SideContext provider wraps the entire React app. A global dropdown in the top navigation allows switching between All / Bride Side / Groom Side. This value is stored in Zustand and passed as a query parameter (?side_id=...) or as a header (X-Side-Id) on all API requests.

### 3.3.3 Side-Aware Service Pattern (Backend)

```
// All service methods accept an optional sideFilter
async getGuests(weddingId: string, sideId?: string, pagination: PaginationDto) {
  return this.prisma.guest.findMany({
    where: {
      wedding_id: weddingId,
      ...(sideId ? { side_id: sideId } : {}),  // scoped or global
    },
    skip: pagination.skip,
    take: pagination.limit,
  });
}
```

## 3.4 Guest Management Module (CORE)

### 3.4.1 Guest Fields

| Field | Type | Required | Notes |
|---|---|---|---|
| id | UUID | ✅ | PK |
| wedding_id | UUID FK | ✅ | → weddings.id |
| side_id | UUID FK | ✅ | → wedding_sides.id |
| group_id | UUID FK | ❌ | → guest_groups.id (family) |
| full_name | VARCHAR | ✅ | Guest full name |
| phone_number | VARCHAR | ✅ | E.164 format e.g. +919876543210 |
| email | VARCHAR | ❌ | |
| relationship | VARCHAR | ❌ | e.g. 'Uncle', 'College Friend' |
| tags | TEXT[] | ❌ | PostgreSQL array: ['VIP','Outstation'] |
| is_primary | BOOLEAN | ❌ | Primary contact for the family group |
| invitation_sent | BOOLEAN | ❌ | Default false |
| notes | TEXT | ❌ | Organiser notes |
| created_at | TIMESTAMPTZ | ✅ | |

### 3.4.2 Guest Groups (Family-Level Tracking)

Multiple guests belong to one guest_group (family unit). RSVP headcount is aggregated at the group level. Example: 'Sharma Family' group contains 4 members; RSVP response covers all 4.

### 3.4.3 Bulk Upload (CSV)

Expected CSV format:

```
full_name,phone_number,family_group,relationship,tags,side
Ramesh Sharma,+919876543210,Sharma Family,Uncle,VIP,bride
Sunita Sharma,+919876543211,Sharma Family,Aunt,,bride
Kiran Mehta,+919845001234,Mehta Family,College Friend,Outstation,groom
```

Bulk upload validation rules:

- Phone number: must be valid E.164 format (validate with libphonenumber-js)
- Duplicate detection: if phone_number already exists in wedding → skip, log as duplicate
- Side field: must match an existing side name in the wedding (bride/groom)
- Max file size: 5MB. Max rows per upload: 2000
- Processing: async via BullMQ queue (csv-import queue) — returns a job_id, frontend polls GET /jobs/:id for status

## 3.5 WhatsApp Campaign Module

### 3.5.1 Message Template Structure

Templates are pre-approved via Meta Business Manager. Variables follow {{1}}, {{2}} notation in Meta's API but are mapped to named variables in the platform:

```
// Template: wedding_invitation_v1
// Meta-registered body:
// 'Dear {{1}}, You are warmly invited to the wedding of {{2}} & {{3}}.
//  Event: {{4}} on {{5}} at {{6}}. Venue: {{7}}.
//  Please confirm your attendance by replying YES or NO.'

// Platform variable mapping (stored in campaigns.template_variables):
{
  '{{1}}': '{{guest.full_name}}',
  '{{2}}': '{{wedding.bride_name}}',
  '{{3}}': '{{wedding.groom_name}}',
  '{{4}}': '{{event.name}}',
  '{{5}}': '{{event.event_date}}',
  '{{6}}': '{{event.event_time}}',
  '{{7}}': '{{event.venue}}'
}
```

### 3.5.2 Campaign Constraints

- Each campaign must be assigned to a side_id (bride / groom) OR marked as both_sides: true
- When both_sides is true, separate message queue jobs are created for each side's guest list
- A campaign can only target guests from its assigned side(s)
- SIDE_MANAGER users can only create campaigns for their assigned side

### 3.5.3 Delivery Status Tracking

| Status | Trigger | Description |
|---|---|---|
| QUEUED | Campaign launched | Message job created in BullMQ |
| SENT | API call succeeded | WhatsApp API accepted the message |
| DELIVERED | Webhook: delivered status | Message reached recipient's device |
| READ | Webhook: read status | Recipient opened the message |
| FAILED | Webhook: failed status / API error | Permanent or temporary failure |
| REPLIED | Incoming webhook message | Guest sent any reply |

## 3.6 RSVP System (Critical Core)

### 3.6.1 RSVP Conversation Flow

```
Organiser sends invitation
      ↓
Guest receives WhatsApp message
      ↓
Guest replies: 'Yes' / 'No' / 'Maybe'
      ↓
Webhook fires → NestJS RSVPService.parseIncoming()
      ↓
Response classified → rsvp_responses record created
      ↓
If YES: follow-up message: 'How many from your family will attend?'
      ↓
Guest replies: '3' or 'Myself and wife' → headcount parsed
      ↓
n8n workflow triggered based on response type
```

### 3.6.2 Response Parser Logic

```
// rsvp.parser.ts
export function classifyResponse(message: string): RSVPStatus {
  const msg = message.toLowerCase().trim();

  const YES_PATTERNS = [/^yes/, /^attending/, /^confirm/, /^\bha\b/, /^will come/];
  const NO_PATTERNS  = [/^no/, /^not coming/, /^can't/, /^cannot/, /^sorry/];
  const MAYBE_PATTERNS = [/^maybe/, /^possibly/, /^not sure/, /^depends/];

  if (YES_PATTERNS.some(p => p.test(msg))) return RSVPStatus.CONFIRMED;
  if (NO_PATTERNS.some(p => p.test(msg)))  return RSVPStatus.DECLINED;
  if (MAYBE_PATTERNS.some(p => p.test(msg))) return RSVPStatus.MAYBE;
  return RSVPStatus.AMBIGUOUS; // triggers human review queue
}

export function extractHeadcount(message: string): number | null {
  const numMatch = message.match(/\b(\d+)\b/);
  if (numMatch) return parseInt(numMatch[1]);
```

```
  // Word-to-number: 'two' → 2, 'three' → 3
  const wordMap: Record<string, number> = {
    one: 1, two: 2, three: 3, four: 4, five: 5,
    six: 6, seven: 7, eight: 8, nine: 9, ten: 10
  };
  const wordMatch = message.toLowerCase().match(
    /\b(one|two|three|four|five|six|seven|eight|nine|ten)\b/
  );
  return wordMatch ? wordMap[wordMatch[1]] : null;
}
```

### 3.6.3 RSVP States

| Status | Meaning | Next Action |
|---|---|---|
| PENDING | Invitation sent, no reply received | n8n reminder after configured days |
| CONFIRMED | Guest replied YES | Ask headcount; send thank-you |
| DECLINED | Guest replied NO | Log; no further messages |
| MAYBE | Guest is unsure | n8n follow-up closer to event |
| AMBIGUOUS | Reply not parseable | Flag for manual review in dashboard |
| NO_REPLY | Exceeded reminder attempts | Mark final; log as no-show risk |

## 3.7 Automation Module (n8n Workflows)

### 3.7.1 Workflow: RSVP Reminder (No Response)

- Trigger: HTTP Webhook (NestJS fires POST /n8n/trigger/rsvp-reminder)
- Input payload: { guest_id, wedding_id, side_id, original_message_sid, days_since_sent }
- Condition node: if days_since_sent >= reminder_after_days (configurable per campaign)
- Action: Call NestJS POST /internal/messages/send-reminder with guest context
- Max retries: 2 (configurable). After max → mark guest as NO_REPLY

### 3.7.2 Workflow: Thank-You on Confirmation

- Trigger: NestJS fires webhook when RSVPStatus becomes CONFIRMED
- Delay node: 30 seconds (avoids immediate auto-reply feel)
- Action: Send thank-you WhatsApp template with event details and venue map link

### 3.7.3 Workflow: Pre-Event Reminder

- Trigger: n8n CRON schedule — runs daily at 9:00 AM
- Action: Query NestJS GET /internal/events/upcoming?days=1,3,7
- For each confirmed guest attending an upcoming event: send reminder template
- Template variables: event name, date, time, venue, maps link

### 3.7.4 n8n ↔ NestJS Integration

NestJS exposes internal endpoints protected by a shared HMAC secret (X-Internal-Secret header):

```
// n8n accesses these NestJS internal routes:
POST /internal/messages/send-reminder     // send a WhatsApp reminder
POST /internal/messages/send-thankyou     // send a thank-you message
GET  /internal/events/upcoming            // fetch upcoming events
GET  /internal/guests/no-response         // guests with no RSVP
PATCH /internal/rsvp/:id/status           // update RSVP status programmatically
```

## 3.8 Dashboard & Analytics Module

### 3.8.1 Dashboard Metrics (Per Side + Global)

| Metric | Query | Displayed As |
|---|---|---|
| Total Invited | COUNT(guests WHERE side_id = ?) | Number card |
| Confirmed | COUNT(rsvp WHERE status = CONFIRMED) | Number card + % |
| Declined | COUNT(rsvp WHERE status = DECLINED) | Number card + % |
| Pending | COUNT(rsvp WHERE status = PENDING) | Number card + % |
| Total Headcount | SUM(rsvp.confirmed_headcount) | Number card |
| Response Rate | (Confirmed+Declined) / Total Invited * 100 | Pie chart |
| Side Comparison | Bride vs Groom RSVP breakdown | Grouped bar chart |
| Family Breakdown | Per group: invited count vs confirmed count | Table with progress bar |
| Campaign Performance | Sent / Delivered / Read / Failed per campaign | Table |

### 3.8.2 Dashboard API Aggregation Strategy

- Use PostgreSQL materialized views refreshed every 5 minutes for heavy aggregations
- For real-time counts (small weddings <500 guests): direct COUNT queries with indices
- For large weddings (>500 guests): background job recalculates stats and stores in wedding_stats_cache table
- API endpoint: GET /stats/dashboard?weddingId=&sideId= returns pre-aggregated JSON

## 3.9 Message Logs Module

| Field | Type | Notes |
|---|---|---|
| id | UUID | PK |
| wedding_id | UUID FK | → weddings.id |
| side_id | UUID FK | → wedding_sides.id |

| Field | Type | Notes |
|---|---|---|
| guest_id | UUID FK | → guests.id |
| campaign_id | UUID FK | → campaigns.id (nullable) |
| direction | ENUM | OUTBOUND \| INBOUND |
| whatsapp_msg_id | VARCHAR | Meta message ID (wamid.xxx) |
| body | TEXT | Message content |
| status | ENUM | QUEUED\|SENT\|DELIVERED\|READ\|FAILED\|REPLIED |
| error_code | VARCHAR | WhatsApp error code if FAILED |
| error_message | TEXT | Human-readable error detail |
| sent_at | TIMESTAMPTZ | |
| delivered_at | TIMESTAMPTZ | From webhook |
| read_at | TIMESTAMPTZ | From webhook |

### 3.10 Settings Module

- WhatsApp API Configuration: phone_number_id, waba_id, access_token (encrypted at rest with AES-256)
- Template Management: list, preview, and map approved Meta templates to platform template slots
- Reminder Configuration: days_before_reminder (default 3), max_reminders (default 2)
- Team Management: invite, remove, change role of team members
- Wedding Details: edit couple names, date, venue, branding colors per side

# 4. UI/UX SCREEN SPECIFICATIONS

### 4.1 Screen: Login / OTP Verification

| Attribute | Detail |
|---|---|
| Purpose | Authenticate user via email + OTP |
| UI Components | Email input, Send OTP button, 6-digit OTP input, Resend timer (60s) |
| User Interactions | Submit email → OTP sent toast → enter OTP → redirect to /dashboard or /setup |
| Validation Rules | Email: valid format. OTP: exactly 6 digits. OTP expiry: error if expired. |

### 4.2 Screen: Wedding Setup Wizard

| Attribute | Detail |
|---|---|
| Purpose | Create new wedding project |
| UI Components | 3-step wizard: (1) Couple details (2) Event details (3) Team invite |
| User Interactions | Step progress bar; Next/Back buttons; skip team invite; final step creates wedding + sides |
| Validation Rules | Bride/Groom name required. Wedding date must be in future. Phone required for team invites. |

### 4.3 Screen: Dashboard (Home)

| Attribute | Detail |
|---|---|
| Purpose | Central analytics overview with side selector |
| UI Components | Top nav with Side Selector (All/Bride/Groom toggle); KPI cards; Pie chart; Bar chart; Recent activity feed |
| User Interactions | Toggle side → all KPIs re-fetch for selected side; Click KPI card → drill-down page |
| Validation Rules | N/A (read-only) |

### 4.4 Screen: Side Selector (Global Component)

| Attribute | Detail |
|---|---|
| Purpose | Switch active side context across all screens |
| UI Components | Dropdown or tab group: All Guests \| Bride Side \| Groom Side; Color-coded by side accent |
| User Interactions | Selection updates Zustand sideContext → all API calls include ?side_id param |
| Validation Rules | SIDE_MANAGER users: side selector is locked to their assigned side |

### 4.5 Screen: Guest List Page

| Attribute | Detail |
|---|---|
| Purpose | View, filter, and manage all guests |
| UI Components | Table with columns: Name, Phone, Family Group, Side badge, Tags, RSVP Status; Search bar; Filters (side, tag, RSVP status); Bulk actions: Assign side, Delete, Export CSV; Add Guest button; Upload CSV button |
| User Interactions | Click row → Guest Detail drawer; Bulk select → action bar appears; Upload → CSV parse preview modal |
| Validation Rules | Bulk assign side requires selection. Delete requires confirmation dialog. |

### 4.6 Screen: Add/Edit Guest Modal

| Attribute | Detail |
|---|---|
| Purpose | Create or edit a single guest |
| UI Components | Form fields: Full Name*, Phone* (with country code selector), Family Group (autocomplete), Relationship, Tags (multi-select chip input), Side (radio: Bride/Groom), Notes; Save / Cancel buttons |
| User Interactions | Phone input validates on blur; Family group creates new group if not found |
| Validation Rules | Name: required, min 2 chars. Phone: E.164 format validation. Side: required. |

### 4.7 Screen: CSV Upload Page

| Attribute | Detail |
|---|---|
| Purpose | Bulk import guests from CSV |
| UI Components | Drag-and-drop upload zone; Template download button; Column mapping UI; Preview table (first 10 rows); Error highlights; Import button |
| User Interactions | Upload → parse columns → map to fields → preview → confirm → async import with progress bar |
| Validation Rules | File type: .csv only. Max size: 5MB. Required columns: full_name, phone_number, side. |

### 4.8 Screen: Campaign Builder

| Attribute | Detail |
|---|---|
| Purpose | Create and configure a WhatsApp campaign |
| UI Components | Campaign name; Side selector (Bride/Groom/Both); Template picker (approved Meta templates); Variable mapping UI; Audience filter (all guests / specific tags / specific family); Test message section; Schedule toggle (send now vs send at time) |
| User Interactions | Select template → variable placeholders appear → map to guest fields → preview live message; Send test → modal to enter test phone number |
| Validation Rules | Template required. At least 1 audience member. Side required. |

### 4.9 Screen: Campaign Launch Confirmation

| Attribute | Detail |
|---|---|
| Purpose | Final review before sending messages |
| UI Components | Summary: campaign name, side, total recipients, template, schedule; Warning banner for bulk sends >200; Confirm & Launch button |
| User Interactions | Launch → job queued → toast with job ID → redirect to Campaign Status page |
| Validation Rules | Cannot launch if WhatsApp API not configured. |

### 4.10 Screen: Campaign Status / Message Logs

| Attribute | Detail |
|---|---|
| Purpose | Track delivery status per campaign |
| UI Components | Status summary cards (Sent/Delivered/Read/Failed); Per-message table with status badge; Retry failed button; Export log button |
| User Interactions | Click message row → full log detail; Filter by status; Retry sends a new message job |
| Validation Rules | Retry: max 3 retries per guest per campaign. |

### 4.11 Screen: RSVP Analytics Page

| Attribute | Detail |
|---|---|
| Purpose | Deep-dive into RSVP responses |
| UI Components | Donut chart (Confirmed/Declined/Pending/Maybe); Headcount summary; Family-level table; Ambiguous responses panel (with manual classify buttons); Export to CSV |
| User Interactions | Filter by side, event, tag; Click family row → expand member list with individual statuses |
| Validation Rules | Manual classification: SIDE_MANAGER and above only. |

### 4.12 Screen: Settings: WhatsApp Config

| Attribute | Detail |
|---|---|
| Purpose | Connect WhatsApp Business API |
| UI Components | Form: Phone Number ID, WhatsApp Business Account ID, Access Token (masked); Test Connection button; Webhook URL display (copy button); Template sync button |
| User Interactions | Save → backend validates API credentials by calling Meta test endpoint; Test → sends test message to organiser's number |
| Validation Rules | All three fields required. Token format validated. |

### 4.13 Screen: Team Management

| Attribute | Detail |
|---|---|
| Purpose | Manage users and their roles |
| UI Components | Team member table: Avatar, Name, Email, Role badge, Assigned Side badge, Actions (Edit Role, Remove); Invite Member button |
| User Interactions | Invite → modal with email, role, side assignment; Edit Role → inline dropdown; Remove → confirmation dialog |

| Attribute | Detail |
|---|---|
| Validation Rules | Cannot remove yourself. Cannot demote last WEDDING_ADMIN. |

# 5. DATABASE SCHEMA

**ORM:** All schemas use Prisma ORM with PostgreSQL 15. UUIDs generated with gen_random_uuid(). All timestamps use TIMESTAMPTZ (UTC). Soft deletes via deleted_at nullable column.

### 5.x Table: users

```
CREATE TABLE users (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email        VARCHAR(255) UNIQUE NOT NULL,
  full_name    VARCHAR(255) NOT NULL,
  phone        VARCHAR(20),
  is_verified  BOOLEAN DEFAULT FALSE,
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  updated_at   TIMESTAMPTZ DEFAULT NOW(),
  deleted_at   TIMESTAMPTZ          -- soft delete
);
CREATE INDEX idx_users_email ON users(email);
```

### 5.x Table: weddings

```
CREATE TABLE weddings (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_slug       VARCHAR(100) UNIQUE NOT NULL,
  bride_name        VARCHAR(255) NOT NULL,
  groom_name        VARCHAR(255) NOT NULL,
  wedding_date      DATE,
  venue             TEXT,
  whatsapp_config   JSONB,           -- encrypted API credentials
  reminder_days     INT DEFAULT 3,   -- days before reminder fires
  max_reminders     INT DEFAULT 2,
  created_by        UUID NOT NULL REFERENCES users(id),
  is_active         BOOLEAN DEFAULT TRUE,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);
```

### 5.x Table: wedding_sides (CRITICAL)

```
CREATE TABLE wedding_sides (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id    UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  name          VARCHAR(20) NOT NULL,   -- 'bride' | 'groom'
  display_name  VARCHAR(100) NOT NULL,
  color_hex     VARCHAR(7) DEFAULT '#0E7490',
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(wedding_id, name)
```

```
);
CREATE INDEX idx_wedding_sides_wedding ON wedding_sides(wedding_id);
```

### 5.x Table: wedding_users (junction)

```
CREATE TABLE wedding_users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id        UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role              VARCHAR(30) NOT NULL, -- WEDDING_ADMIN|SIDE_MANAGER|VIEWER
  assigned_side_ids UUID[],               -- NULL = all sides
  invited_by        UUID REFERENCES users(id),
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(wedding_id, user_id)
);
```

### 5.x Table: events

```
CREATE TABLE events (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id   UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  side_id      UUID REFERENCES wedding_sides(id),  -- NULL = both sides
  name         VARCHAR(255) NOT NULL,
  event_date   DATE,
  event_time   TIME,
  venue        TEXT,
  maps_link    TEXT,
  dress_code   VARCHAR(100),
  notes        TEXT,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_events_wedding ON events(wedding_id);
CREATE INDEX idx_events_date    ON events(event_date);
```

### 5.x Table: guest_groups

```
CREATE TABLE guest_groups (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id   UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  side_id      UUID NOT NULL REFERENCES wedding_sides(id),
  name         VARCHAR(255) NOT NULL,  -- 'Sharma Family'
  notes        TEXT,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_groups_wedding ON guest_groups(wedding_id);
CREATE INDEX idx_groups_side    ON guest_groups(side_id);
```

### 5.x Table: guests

```
CREATE TABLE guests (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id        UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  side_id           UUID NOT NULL REFERENCES wedding_sides(id),
  group_id          UUID REFERENCES guest_groups(id),
  full_name         VARCHAR(255) NOT NULL,
  phone_number      VARCHAR(20) NOT NULL,
  email             VARCHAR(255),
```

```
  relationship     VARCHAR(100),
  tags             TEXT[] DEFAULT '{}',
  is_primary       BOOLEAN DEFAULT FALSE,
  invitation_sent  BOOLEAN DEFAULT FALSE,
  notes            TEXT,
  created_at       TIMESTAMPTZ DEFAULT NOW(),
  updated_at       TIMESTAMPTZ DEFAULT NOW(),
  deleted_at       TIMESTAMPTZ,
  UNIQUE(wedding_id, phone_number)
);
CREATE INDEX idx_guests_wedding  ON guests(wedding_id);
CREATE INDEX idx_guests_side     ON guests(side_id);
CREATE INDEX idx_guests_group    ON guests(group_id);
CREATE INDEX idx_guests_phone    ON guests(phone_number);
CREATE INDEX idx_guests_tags     ON guests USING GIN(tags);
```

## 5.x Table: campaigns

```
CREATE TABLE campaigns (
  id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id         UUID NOT NULL REFERENCES weddings(id) ON DELETE CASCADE,
  side_id            UUID REFERENCES wedding_sides(id), -- NULL = both
  both_sides         BOOLEAN DEFAULT FALSE,
  name               VARCHAR(255) NOT NULL,
  template_name      VARCHAR(255) NOT NULL, -- Meta template name
  template_variables JSONB NOT NULL,        -- variable mapping
  audience_filter    JSONB,                 -- tags, groups, etc.
  status             VARCHAR(30) DEFAULT 'DRAFT',
                     -- DRAFT|SCHEDULED|RUNNING|COMPLETED|FAILED
  scheduled_at       TIMESTAMPTZ,
  launched_at        TIMESTAMPTZ,
  completed_at       TIMESTAMPTZ,
  created_by         UUID NOT NULL REFERENCES users(id),
  created_at         TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_campaigns_wedding ON campaigns(wedding_id);
CREATE INDEX idx_campaigns_side    ON campaigns(side_id);
```

## 5.x Table: messages

```
CREATE TABLE messages (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id       UUID NOT NULL REFERENCES weddings(id),
  side_id          UUID NOT NULL REFERENCES wedding_sides(id),
  guest_id         UUID REFERENCES guests(id),
  campaign_id      UUID REFERENCES campaigns(id),
  direction        VARCHAR(10) NOT NULL,  -- OUTBOUND|INBOUND
  whatsapp_msg_id  VARCHAR(255) UNIQUE,   -- wamid from Meta
  body             TEXT,
  status           VARCHAR(20) DEFAULT 'QUEUED',
  error_code       VARCHAR(50),
  error_message    TEXT,
  retry_count      INT DEFAULT 0,
  sent_at          TIMESTAMPTZ,
  delivered_at     TIMESTAMPTZ,
  read_at          TIMESTAMPTZ,
  created_at       TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_messages_wedding   ON messages(wedding_id);
```

```
CREATE INDEX idx_messages_guest     ON messages(guest_id);
CREATE INDEX idx_messages_campaign  ON messages(campaign_id);
CREATE INDEX idx_messages_wamid     ON messages(whatsapp_msg_id);
CREATE INDEX idx_messages_status    ON messages(status);
```

### 5.x Table: rsvp_responses

```
CREATE TABLE rsvp_responses (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  wedding_id          UUID NOT NULL REFERENCES weddings(id),
  side_id             UUID NOT NULL REFERENCES wedding_sides(id),
  guest_id            UUID NOT NULL REFERENCES guests(id),
  group_id            UUID REFERENCES guest_groups(id),
  campaign_id         UUID REFERENCES campaigns(id),
  status              VARCHAR(20) NOT NULL DEFAULT 'PENDING',
                      -- PENDING|CONFIRMED|DECLINED|MAYBE|AMBIGUOUS|NO_REPLY
  confirmed_headcount INT DEFAULT 0,
  attendee_names      TEXT[],           -- optional names from guest
  raw_response        TEXT,             -- original WhatsApp message
  response_message_id UUID REFERENCES messages(id),
  reminder_count      INT DEFAULT 0,
  last_reminded_at    TIMESTAMPTZ,
  responded_at        TIMESTAMPTZ,
  notes               TEXT,             -- manual override notes
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(guest_id, campaign_id)         -- one RSVP per guest per campaign
);
CREATE INDEX idx_rsvp_wedding  ON rsvp_responses(wedding_id);
CREATE INDEX idx_rsvp_side     ON rsvp_responses(side_id);
CREATE INDEX idx_rsvp_guest    ON rsvp_responses(guest_id);
CREATE INDEX idx_rsvp_status   ON rsvp_responses(status);
```

# 6. API DESIGN

Base URL: https://api.weddingcrm.app/v1 | All endpoints require Authorization: Bearer <access_token> unless marked [public]

## 6.1 Authentication APIs

### POST /auth/request-otp

| Attribute | Detail |
|---|---|
| Method + URL | POST /auth/request-otp |
| Auth Required | Public |
| Request Body / Params | { "email": "george@example.com" } |
| Response Example | { "message": "OTP sent to email", "expires_in": 600 } |
| Notes | Rate limited: 5 requests/15min per IP |

### POST /auth/verify-otp

| Attribute | Detail |
|---|---|
| Method + URL | POST /auth/verify-otp |
| Auth Required | Public |
| Request Body / Params | { "email": "george@example.com", "otp": "482910" } |
| Response Example | { "access_token": "eyJ...", "refresh_token": "...", "user": { "id": "...", "email": "..." } } |
| Notes | OTP expires in 10 min. Max 3 wrong attempts → lock 15 min |

### POST /auth/refresh

| Attribute | Detail |
|---|---|
| Method + URL | POST /auth/refresh |
| Auth Required | Refresh token (cookie) |
| Request Body / Params | N/A (token in HttpOnly cookie) |
| Response Example | { "access_token": "eyJ..." } |
| Notes | — |

### POST /auth/logout

| Attribute | Detail |
|---|---|
| Method + URL | POST /auth/logout |
| Auth Required | Bearer |
| Request Body / Params | {} |
| Response Example | { "message": "Logged out" } |
| Notes | Invalidates refresh token in Redis |

## 6.2 Wedding APIs

### POST /weddings

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings |
| Auth Required | Bearer |
| Request Body / Params | { "bride_name": "Priya", "groom_name": "Rahul", "wedding_date": "2025-12-15", "venue": "Grand Ballroom, Trivandrum" } |

| Attribute | Detail |
|---|---|
| Response Example | { "id": "uuid", "tenant_slug": "priya-rahul-2025", "sides": [ {"id":"uuid","name":"bride"}, {"id":"uuid","name":"groom"} ] } |
| Notes | Auto-creates bride + groom sides |

### GET /weddings/:id

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id |
| Auth Required | Bearer |
| Request Body / Params | N/A |
| Response Example | { "id": "...", "bride_name": "...", "sides": [...], "events": [...] } |
| Notes | — |

### PATCH /weddings/:id

| Attribute | Detail |
|---|---|
| Method + URL | PATCH /weddings/:id |
| Auth Required | Bearer (WEDDING_ADMIN) |
| Request Body / Params | { "venue": "Updated Venue", "reminder_days": 5 } |
| Response Example | { "id": "...", updated fields... } |
| Notes | — |

## 6.3 Wedding Sides APIs

### GET /weddings/:id/sides

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/sides |
| Auth Required | Bearer |
| Request Body / Params | N/A |
| Response Example | [ { "id": "uuid", "name": "bride", "display_name": "Bride Side", "color_hex": "#E91E8C", "guest_count": 214 }, ... ] |
| Notes | Includes computed guest_count |

### PATCH /weddings/:id/sides/:sideId

| Attribute | Detail |
|---|---|
| Method + URL | PATCH /weddings/:id/sides/:sideId |
| Auth Required | Bearer (WEDDING_ADMIN) |
| Request Body / Params | { "display_name": "Team Bride", "color_hex": "#E91E8C" } |
| Response Example | { "id": "...", "display_name": "Team Bride" } |
| Notes | — |

## 6.4 Guest APIs

### GET /weddings/:id/guests

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/guests |
| Auth Required | Bearer |
| Request Body / Params | Query params: ?sideId=&groupId=&tag=&status=&search=&page=1&limit=50 |
| Response Example | { "data": [ {...guest} ], "total": 320, "page": 1, "limit": 50 } |
| Notes | SideGuard enforces side restriction for SIDE_MANAGER |

### POST /weddings/:id/guests

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/guests |
| Auth Required | Bearer |
| Request Body / Params | { "full_name": "Ramesh Sharma", "phone_number": "+919876543210", "side_id": "uuid", "group_id": "uuid", "tags": ["VIP"], "relationship": "Uncle" } |
| Response Example | { "id": "uuid", "full_name": "Ramesh Sharma", ... } |
| Notes | Returns 409 if phone already exists in wedding |

### PATCH /weddings/:id/guests/:guestId

| Attribute | Detail |
|---|---|
| Method + URL | PATCH /weddings/:id/guests/:guestId |
| Auth Required | Bearer |
| Request Body / Params | { "tags": ["VIP", "Outstation"], "notes": "Needs veg meal" } |
| Response Example | { "id": "...", updated fields... } |
| Notes | — |

### DELETE /weddings/:id/guests/:guestId

| Attribute | Detail |
|---|---|
| Method + URL | DELETE /weddings/:id/guests/:guestId |
| Auth Required | Bearer (WEDDING_ADMIN) |
| Request Body / Params | N/A |
| Response Example | { "message": "Guest deleted" } |
| Notes | Soft delete (sets deleted_at) |

### POST /weddings/:id/guests/bulk

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/guests/bulk |
| Auth Required | Bearer |
| Request Body / Params | multipart/form-data: file=<csv>, side_id=<uuid> |
| Response Example | { "job_id": "uuid", "status": "QUEUED", "poll_url": "/jobs/uuid" } |
| Notes | Async. Returns job ID. Max 2000 rows per upload. |

### POST /weddings/:id/guests/bulk-assign-side

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/guests/bulk-assign-side |
| Auth Required | Bearer (WEDDING_ADMIN) |
| Request Body / Params | { "guest_ids": ["uuid1","uuid2"], "side_id": "uuid" } |
| Response Example | { "updated": 2, "failed": 0 } |
| Notes | — |

## 6.5 Campaign APIs

### POST /weddings/:id/campaigns

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/campaigns |
| Auth Required | Bearer |
| Request Body / Params | { "name": "Main Invitation - Bride Side", "side_id": "uuid", "template_name": "wedding_invitation_v1", "template_variables": { "{{1}}": "{{guest.full_name}}", ... }, "audience_filter": { "tags": ["VIP"], "all": false } } |

| Attribute | Detail |
|---|---|
| Response Example | { "id": "uuid", "status": "DRAFT" } |
| Notes | SideGuard prevents SIDE_MANAGER from creating campaign for opposing side |

### POST /weddings/:id/campaigns/:campId/launch

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/campaigns/:campId/launch |
| Auth Required | Bearer |
| Request Body / Params | { "scheduled_at": null } |
| Response Example | { "campaign_id": "uuid", "status": "RUNNING", "total_recipients": 150, "job_ids": ["..."] } |
| Notes | Sets status to RUNNING, enqueues BullMQ jobs |

### POST /weddings/:id/campaigns/:campId/test

| Attribute | Detail |
|---|---|
| Method + URL | POST /weddings/:id/campaigns/:campId/test |
| Auth Required | Bearer |
| Request Body / Params | { "test_phone": "+919876543210" } |
| Response Example | { "message": "Test message sent" } |
| Notes | Sends single test message without creating message log records |

### GET /weddings/:id/campaigns/:campId/stats

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/campaigns/:campId/stats |
| Auth Required | Bearer |
| Request Body / Params | N/A |
| Response Example | { "sent": 150, "delivered": 147, "read": 89, "failed": 3, "replied": 62 } |
| Notes | — |

## 6.6 WhatsApp Webhook APIs

### GET /webhook/whatsapp

| Attribute | Detail |
|---|---|
| Method + URL | GET /webhook/whatsapp |
| Auth Required | Public (Meta verification) |
| Request Body / Params | Query: ?hub.mode=subscribe&hub.verify_token=SECRET&hub.challenge=CHALLENGE |
| Response Example | Returns hub.challenge value as plain text |
| Notes | Meta webhook verification handshake. MUST be implemented exactly. |

### POST /webhook/whatsapp

| Attribute | Detail |
|---|---|
| Method + URL | POST /webhook/whatsapp |
| Auth Required | Public (HMAC signature validated) |
| Request Body / Params | Meta webhook payload (see Section 7 for full structure) |
| Response Example | { "status": "ok" } |
| Notes | Must respond 200 within 5 seconds. Heavy processing enqueued async. |

## 6.7 RSVP APIs

### GET /weddings/:id/rsvp

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/rsvp |
| Auth Required | Bearer |
| Request Body / Params | Query: ?sideId=&status=&groupId=&page=1&limit=50 |
| Response Example | { "data": [ { "guest": {...}, "status": "CONFIRMED", "confirmed_headcount": 3, "responded_at": "..." } ], "total": 200 } |
| Notes | — |

### PATCH /weddings/:id/rsvp/:rsvpId

| Attribute | Detail |
|---|---|
| Method + URL | PATCH /weddings/:id/rsvp/:rsvpId |
| Auth Required | Bearer |
| Request Body / Params | { "status": "CONFIRMED", "confirmed_headcount": 2, "notes": "Manually confirmed via call" } |
| Response Example | { "id": "...", updated... } |

| Attribute | Detail |
|---|---|
| Notes | Manual override by organiser |

## 6.8 Dashboard Stats APIs

### GET /weddings/:id/stats/dashboard

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/stats/dashboard |
| Auth Required | Bearer |
| Request Body / Params | Query: ?sideId= (optional) |
| Response Example | { "total_invited": 450, "confirmed": 210, "declined": 80, "pending": 140, "maybe": 20, "total_headcount": 580, "response_rate": 64.4, "by_side": [ { "side": "bride", "invited": 220, "confirmed": 100, "headcount": 280 }, { "side": "groom", "invited": 230, "confirmed": 110, "headcount": 300 } ] } |
| Notes | Served from stats cache for weddings >500 guests |

### GET /weddings/:id/stats/families

| Attribute | Detail |
|---|---|
| Method + URL | GET /weddings/:id/stats/families |
| Auth Required | Bearer |
| Request Body / Params | Query: ?sideId= |
| Response Example | { "data": [ { "group_name": "Sharma Family", "total_members": 4, "confirmed": 3, "headcount": 3 } ] } |
| Notes | — |

# 7. WHATSAPP INTEGRATION

## 7.1 Message Sending (Meta Cloud API)

```
// whatsapp.service.ts
async sendTemplate(to: string, templateName: string, variables: string[], config:
WaConfig) {
  const payload = {
    messaging_product: 'whatsapp',
    to,
    type: 'template',
    template: {
```

```
      name: templateName,
      language: { code: 'en_IN' },
      components: [{
        type: 'body',
        parameters: variables.map(v => ({ type: 'text', text: v }))
      }]
    }
  };

  const response = await axios.post(
    `https://graph.facebook.com/v18.0/${config.phone_number_id}/messages`,
    payload,
    { headers: { Authorization: `Bearer ${config.access_token}` } }
  );
  return response.data.messages[0].id; // returns wamid
}
```

## 7.2 Incoming Webhook Payload (Full Structure)

```
// POST /webhook/whatsapp — incoming message from guest
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "WABA_ID",
    "changes": [{
      "value": {
        "messaging_product": "whatsapp",
        "metadata": { "phone_number_id": "PHONE_ID" },
        "messages": [{
          "id": "wamid.xxx",
          "from": "919876543210",  // sender phone (no +)
          "timestamp": "1700000000",
          "type": "text",
          "text": { "body": "Yes, we will be attending!" }
        }],
        "statuses": [{
          "id": "wamid.xxx",
          "status": "delivered",  // or "read" | "sent" | "failed"
          "timestamp": "...",
          "recipient_id": "91..."
        }]
      }
    }]
  }]
}
```

## 7.3 Webhook Handler Logic

```
// webhook.controller.ts
@Post('whatsapp')
async handleWebhook(@Body() body: any, @Headers('x-hub-signature-256') sig: string) {
  // 1. Validate HMAC signature
  const isValid = this.webhookService.validateSignature(body, sig);
  if (!isValid) throw new UnauthorizedException();

  // 2. Must return 200 immediately — process async
  this.webhookQueue.add('process-webhook', body); // BullMQ
```

```
  return { status: 'ok' };
}

// webhook.processor.ts (BullMQ Worker)
@Process('process-webhook')
async process(job: Job) {
  const { messages, statuses } = job.data.entry[0].changes[0].value;

  // Handle delivery status updates
  if (statuses) {
    for (const s of statuses) {
      await this.messageService.updateStatus(s.id, s.status, s.timestamp);
    }
  }

  // Handle incoming messages (RSVP replies)
  if (messages) {
    for (const m of messages) {
      const phone = '+' + m.from;
      const guest = await this.guestService.findByPhone(phone);
      if (guest) {
        await this.rsvpService.processIncomingMessage(guest, m.text.body, m.id);
      }
    }
  }
}
```

## 7.4 Error Handling & Rate Limiting

| Error | Code | Handling Strategy |
|---|---|---|
| Invalid access token | 190 | Alert WEDDING_ADMIN via email; pause campaign; do not retry |
| Template not approved | 132001 | Log error; mark campaign FAILED; notify organiser |
| Recipient phone invalid | 131030 | Mark guest as INVALID_PHONE; skip all retries |
| Rate limit exceeded | 80007 | Exponential backoff: 1min → 5min → 30min; max 3 retries |
| Message not deliverable | 131047 | Mark as FAILED; log; include in failed exports |
| Network timeout | N/A | Retry after 2 min; max 3 retries via BullMQ backoff config |

**RATE LIMIT:** Meta WhatsApp Cloud API allows 80 messages/second per phone number in Tier 1 (1000 unique users/day). For large weddings, BullMQ rate limiter is set to 50 messages/sec with concurrency=5 workers. Upgrade to Tier 2 (10,000/day) before launching campaigns >500 guests.

# 8. N8N WORKFLOW DESIGN

## 8.1 Workflow Architecture

n8n is self-hosted alongside the application stack (Docker Compose). NestJS communicates with n8n via HTTP webhooks (n8n Webhook Trigger nodes). n8n calls back to NestJS via HTTP Request nodes authenticated with X-Internal-Secret.

## 8.2 Workflow 1: RSVP No-Response Reminder

```
Trigger:  NestJS HTTP POST → n8n webhook /rsvp-reminder
Payload:  { guest_id, wedding_id, side_id, campaign_id, phone, full_name,
days_since_sent }

Nodes:
  [1] Webhook Trigger — receives payload from NestJS
  [2] IF node — condition: days_since_sent >= {{wedding.reminder_days}}
       TRUE → continue
       FALSE → stop (no action)
  [3] HTTP Request → GET NestJS /internal/rsvp?guestId={{guest_id}}
       Check if still PENDING (guest may have already replied)
  [4] IF node — condition: rsvp.status == 'PENDING'
       TRUE → continue
       FALSE → stop
  [5] IF node — condition: rsvp.reminder_count < wedding.max_reminders
       TRUE → send reminder
       FALSE → PATCH /internal/rsvp/:id { status: 'NO_REPLY' }
  [6] HTTP Request → POST NestJS /internal/messages/send-reminder
       { guest_id, template: 'rsvp_reminder_v1', variables: [...] }
  [7] HTTP Request → PATCH /internal/rsvp/:id { reminder_count: +1, last_reminded_at:
now }
```

## 8.3 Workflow 2: Confirmation Thank-You

```
Trigger:  NestJS HTTP POST → n8n webhook /rsvp-confirmed
Payload:  { guest_id, wedding_id, full_name, confirmed_headcount, events: [...] }

Nodes:
  [1] Webhook Trigger
  [2] Wait node — 30 seconds (humanises the auto-reply)
  [3] HTTP Request → POST NestJS /internal/messages/send-thankyou
       Template: 'rsvp_thankyou_v1'
       Variables: [guest.full_name, confirmed_headcount, event.name,
                   event.date, event.venue, event.maps_link]
```

## 8.4 Workflow 3: Pre-Event Day-Of Reminder (CRON)

```
Trigger:  CRON — every day at 09:00 AM IST (0 3 30 * * ? for UTC+5:30)

Nodes:
  [1] CRON Trigger
  [2] HTTP Request → GET NestJS /internal/events/upcoming?days=1
       Returns: [ { event_id, wedding_id, name, date, venue, maps_link, guests: [...]
} ]
  [3] IF node — any events found?
  [4] SplitInBatches — process up to 50 guests per batch
  [5] HTTP Request → POST NestJS /internal/messages/send-reminder
       Template: 'event_reminder_v1'
  [6] Wait node — 100ms between batches (rate limit protection)
```

# 9. EDGE CASES & ERROR HANDLING

| Edge Case | Scenario | Handling |
|---|---|---|
| Duplicate Guest | Same phone uploaded twice (manual or CSV) | POST /guests returns HTTP 409 with { error: 'DUPLICATE_PHONE', existing_guest_id: '...' }. CSV import skips duplicates and logs them in the import report. |
| Invalid Phone Number | Non-E.164 format, landline, or fake number in CSV | libphonenumber-js validation on import. Invalid numbers logged as INVALID_PHONE in guest record. Excluded from all campaigns. Shown in Guest List with warning badge. |
| Guest Without Side | Guest added without assigning to bride or groom side | Not possible via API — side_id is required field. CSV upload: row with empty side column fails validation and is listed in import errors. UI Add Guest form: side is required radio group. |
| Partial Family Confirmation | Family of 5; only primary guest replies; others don't | RSVP is tracked per guest, not per group. Group shows partial confirmation status. Dashboard highlights groups with mixed statuses. Organiser can manually override group members. |
| Ambiguous RSVP Response | Guest replies 'Coming with family' — no clear YES/NO | Parser returns AMBIGUOUS. RSVP status set to AMBIGUOUS. Shown in RSVP Analytics as needing review. Organiser can manually classify. No automated follow-up until classified. |
| WhatsApp Message Failure | Guest's number not on WhatsApp or blocked | Webhook delivers failed status. Message record updated to FAILED with error_code. After 3 failures: guest marked as WHATSAPP_UNAVAILABLE. Shown in failed recipients export. |
| Large CSV Upload (>1000 rows) | Performance risk on synchronous processing | CSV processing is always async via BullMQ. Frontend polls GET /jobs/:id every 3 seconds. Job returns progress (percent), errors array, and final summary on completion. |
| SIDE_MANAGER accessing wrong side | URL manipulation to access bride data if assigned groom | SideGuard on every protected endpoint. JWT contains assigned_side_ids[]. Guard throws 403 ForbiddenException before any database query executes. |
| WhatsApp API Token Expiry | Meta access tokens can expire | Webhook 190 error triggers: (1) campaign paused, (2) WEDDING_ADMIN emailed, (3) Settings page shows red alert banner. All pending |

| Edge Case | Scenario | Handling |
|---|---|---|
| | | messages held in queue until token refreshed. |
| Concurrent Campaign Launches | Two organizers launch campaigns simultaneously | Idempotency: campaign status transitions guarded by database row-level lock (SELECT FOR UPDATE). Second launch returns 409 if already RUNNING. |

# 10. SCALABILITY DESIGN

## 10.1 Multi-Tenant Architecture

- Each wedding is a tenant. Row-level isolation enforced via wedding_id foreign key on every table.
- No separate database per tenant — shared database with indexed tenant columns for cost efficiency.
- Tenant slug (e.g. priya-rahul-2025) used in JWT payload; all service calls scoped to it.

## 10.2 Queue System (BullMQ + Redis)

| Queue Name | Purpose | Concurrency | Rate Limit |
|---|---|---|---|
| whatsapp-send | Outbound WhatsApp messages | 5 workers | 50 jobs/sec |
| webhook-process | Inbound webhook events | 10 workers | None |
| csv-import | Bulk guest CSV processing | 3 workers | None |
| stats-refresh | Recompute dashboard stats cache | 1 worker | None |
| email-notify | Organiser email notifications | 2 workers | None |

## 10.3 Database Performance (1000+ Guests)

- All foreign key columns indexed (wedding_id, side_id, guest_id, campaign_id)
- RSVP status column indexed — dashboard queries run against index, not full table scan
- Guest tags use GIN index on TEXT[] for fast tag-based filtering
- Dashboard stats materialized view refreshed every 5 minutes via pg_cron
- Pagination enforced on all list endpoints (max 200 per page)
- Full-text search on guest name via PostgreSQL tsvector with ts_rank

# 11. SECURITY

## 11.1 Authentication & Token Security

- Access tokens: JWT, RS256 signing, 15-minute expiry
- Refresh tokens: 256-bit random string, stored as bcrypt hash in Redis with 7-day TTL
- Access token stored in memory (never localStorage to prevent XSS theft)
- Refresh token in HttpOnly, Secure, SameSite=Strict cookie

## 11.2 API Security

| Mechanism | Implementation |
|---|---|
| CORS | Whitelist production domain only. No wildcard origins. |
| Rate Limiting | Global: 100 req/min per IP (NestJS ThrottlerGuard). Auth endpoints: 5 req/15min. |
| Input Validation | class-validator DTOs on all endpoints. Reject unknown properties. |
| SQL Injection | Prisma ORM parameterized queries — no raw SQL with user input. |
| XSS Protection | API returns JSON only. Helmet.js sets Content-Security-Policy headers. |
| WhatsApp Webhook | HMAC-SHA256 signature validation on every incoming webhook request. |
| Internal API Secret | n8n → NestJS internal routes require X-Internal-Secret header (256-bit random). |
| Secrets at Rest | WhatsApp access_token and API keys encrypted with AES-256-GCM before PostgreSQL storage. |

## 11.3 Data Privacy

- Guest phone numbers are PII — access restricted to authenticated team members of the associated wedding only
- SIDE_MANAGER cannot access guest phone numbers from the opposing side
- CSV exports include audit log (who exported, when, how many records)
- Data retention: wedding data auto-archived 90 days after wedding_date (configurable)
- GDPR-ready: DELETE /weddings/:id triggers cascade delete of all guest data with confirmed audit trail

# 12. FUTURE EXTENSIONS

| Feature | Description | Complexity |
|---|---|---|
| AI Response Parsing | Replace regex parser with fine-tuned LLM (GPT-4o or Claude) for multi-language RSVP parsing. Handles Malayalam, Hindi, Tamil responses natively. | High |

| Feature | Description | Complexity |
|---|---|---|
| Voice RSVP | Guest calls a number, IVR collects RSVP via DTMF (1=Yes, 2=No). Twilio Voice API integration. Useful for elderly guests without WhatsApp. | High |
| Multi-Language Support | Campaign templates in Malayalam, Hindi, Tamil. i18n in React (react-intl). Backend locale parameter on message send. | Medium |
| Wedding Microsite | Auto-generated wedding website (priya-rahul-2025.weddingcrm.app). Guests can RSVP via web form as alternative to WhatsApp. | Medium |
| Seating Planner | Table assignment module. Import confirmed headcount into seating chart. Export per-table guest lists. | Medium |
| Vendor Integration | Share confirmed headcount with caterer via API. Auto-generate catering order sheet based on confirmed RSVP. | Medium |
| QR Code Invitation | Generate unique QR per guest. Scanning QR on event day marks attendance in real time. | Low |
| Mobile App | React Native app for day-of attendance tracking. Works offline; syncs when connectivity restored. | High |

**— END OF TECHNICAL SPECIFICATION DOCUMENT —**

*Wedding Guest Management & RSVP Automation Platform · v1.0 · Cifar Systems*